Astroarch Interface

Remote control of an AstroArch observatory from Android

*

User Manual & Installation Guide

Version 0.2.13

Author: Zarletti-Osservatorio Jupiter

May 11, 2026

1. Introduction

Astroarch Interface is an Android application that lets you fully control an astronomical observatory based on the **AstroArch** distribution (Arch Linux for Raspberry Pi with KStars/Ekos/INDI) running on a Raspberry Pi 5. The app connects to the Raspberry Pi via Tailscale and mirrors all Ekos features in real time using a mobile-friendly UI.

The project is split in two parts:

- **astroarch-bridge backend**: a Python daemon (FastAPI + WebSocket) installed on the Raspberry Pi 5. It talks to the INDI server, PHD2 and Ekos via DBus, exposing a REST API + two WebSocket streams to the app.
- **Android app**: written in Flutter, with 14 dedicated screens covering all the Ekos modules cloned for mobile use.

1.1 What you can do

- Real-time view of mount, camera, focuser, filter wheel, dome and weather state (live telemetry over WebSocket)
- Plan and run multi-job capture sequences just like in Ekos
- Run a full pre-flight pipeline: slew → plate solve → sync → guide → autofocus → capture, in one tap
- Search objects by name (SIMBAD) and GoTo with one tap
- Control PHD2 (calibration, guiding, dither) with a live RMS chart
- Control the focuser manually or trigger an iterative autofocus with a live V-curve plot
- Run plate solving via solve-field/astrometry.net and sync the mount on the result
- View captured images live (FITS auto-stretched to JPEG over WebSocket)
- Edit any INDI property from the panel that clones the Ekos INDI Control Panel
- Plan scheduler jobs with twilight/altitude/weather conditions

1.2 Architecture

High-level data flow:

```
Android app --Tailscale (WireGuard)--> Raspberry Pi 5
|- astroarch-bridge :8765
| |- REST /api/*
| |- WS /ws/state (live clone)
| |- WS /ws/frames (JPEG live)
|- INDI server :7624
|- PHD2 :4400 (JSON-RPC)
|- KStars/Ekos (DBus)
|- ~/Pictures/Ekos (FITS storage)
```

Note: Tailscale provides end-to-end WireGuard encryption. There is no need to set up HTTPS on top: port 8765 is exposed in cleartext but the Tailscale tunnel makes the traffic inaccessible to anyone outside your tailnet.

2. Installation

2.1 Prerequisites

On the Raspberry Pi (server side):

- AstroArch (Arch Linux for Raspberry Pi) distribution
- Python 3.11+ with pip
- Tailscale installed and active
- KStars/Ekos installed with a configured INDI profile
- PHD2 installed (optional, for guiding)
- astrometry.net + index files in `~/.local/share/kstars/astrometry/` (optional, for plate solving)

On the Android phone (client side):

- Android 8.0 (API 26) or later
- Tailscale app installed and connected to the same tailnet as the Raspberry Pi
- ~50 MB of free space

2.2 Backend installation on the Raspberry Pi

Step 1 - Transfer the backend

From the computer where you have the *backend/* folder of the Astroarch Interface distribution:

```
scp -r backend/ astronaut@100.74.22.40:/tmp/
```

(Replace `100.74.22.40` with the Tailscale IP of your Raspberry Pi)

Step 2 - Install the service

SSH into the Raspberry Pi and run:

```
ssh astronaut@100.74.22.40
cd /tmp/backend
sudo bash deploy/install.sh --user astronaut
```

The installer:

- Creates/uses the specified user (default: *astroarch*; pass `--user astronaut` to use your own)
- Installs the Python dependencies (FastAPI, uvicorn, astropy, Pillow, watchdog, pydantic, websockets, numpy)
- Installs the `astroarch_bridge` package
- Creates the `astroarch-bridge.service` systemd unit
- Enables auto-start on boot
- Starts the daemon

At the end the script prints the URL and the auto-generated **Bearer token**. Save it: you will need it to connect from the app.

```
==> astroarch-bridge installed and running
URL: http://100.74.22.40:8765
Token: kJ3xSn9mZ7TqW... (example)
```

Step 3 - Verify the service

```
systemctl --user status astroarch-bridge
journalctl --user -u astroarch-bridge -f
curl http://localhost:8765/healthz
```

The healthz output should show:

```
{"ok":true,"version":"0.1.0","indi":"...","phd2":"..."}
```

2.3 Desktop dashboard installation

A small Tk dashboard is installed on the AstroArch desktop to monitor the bridge state and generate the QR code for the app:

```
scp -r desktop_dashboard/
astronaut@100.74.22.40:/home/astronaut/astroarch-bridge-dashboard
scp desktop_dashboard/AstroarchBridge.desktop \
astronaut@100.74.22.40:/home/astronaut/Desktop/
ssh astronaut@100.74.22.40 'chmod +x ~/Desktop/AstroarchBridge.desktop'
```

On the AstroArch desktop you will see the **Astroarch Bridge** icon. It opens a window with service status, Ekos info, URL/Token, a QR code for the app, and Connect/Disconnect buttons.

2.4 APK installation on Android

On the phone:

- Transfer the `AstroarchInterface-v0.2.13.apk` file to the phone (USB / Drive / Tailscale Drop)
- Open the file from the file manager. Android will ask you to enable "Install apps from unknown sources" for the file manager: confirm.
- Tap **Install** → the app shows up in the launcher as **"Astroarch Interface"**

2.5 First connection

Open the app:

- Tap **SCAN QR FROM DASHBOARD** and frame the QR code shown on the desktop dashboard of the Raspberry Pi: host, port and token will be filled in automatically.
- Or fill them in manually:

HOST	Tailscale IP of the Raspberry Pi (e.g. 100.74.22.40)
PORT	8765
TOKEN	The one printed by the installer or read with cat ~/.config/astroarch-bridge/token

Tap **CONNECT**. If the connection works you go straight to the Dashboard. If it fails, tap **TEST** for step-by-step diagnostics.

Warning: If the connection fails with a timeout, make sure Tailscale is active on the phone. Some Android devices (Xiaomi/MIUI, OnePlus, Samsung) need battery optimization disabled for Tailscale, otherwise the VPN gets killed in the background.

3. Main screens

The app uses a 5-tab **bottom navigation** always visible (Dashboard, Mount, Align, Capture, Guide) and a **side drawer** with the advanced screens, opened from the menu icon in the AppBar.

3.1 Dashboard

The first screen after login. Shows in real time:

- **Connection banner** at the top with INDI / PHD2 / WS state / WS frames (green=ok, yellow=connecting, red=fail)
- **Active target** with RA/Dec coordinates and mount status
- **Last captured image preview** auto-stretched, tap for full-screen Live View
- 4 status cards: Mount (tracking), Camera (temperature), Guide (PHD2 RMS), Focuser (position)
- Observatory telemetry: weather, dome, weather safe

3.2 Mount

Full telescope control:

- **Live RA/Dec** updated in real time, pier side, current mount status
- **SIMBAD search**: type a name (M 31, NGC 7000, Vega) and tap SEARCH. RA/Dec resolved via astropy. Buttons: GOTO+TRACK / SLEW / SYNC
- **Manual GoTo** with RA (hours) and Dec (degrees) fields
- **N/S/E/W slew joypad** with rate selection (GUIDE / CENTERING / FIND / MAX depending on the driver)
- Quick Park/Unpark/Sync/Stop
- **Tracking mode** chips: Sidereal / Lunar / Solar / Off

3.3 Align (Plate Solve + Polar Align)

Plate Solve tab — exact clone of Ekos Align:

- Live FITS preview (pinch zoom) **stretched identically to Ekos**
- Configurable Exposure / Gain / Binning, applied to the camera before the solve
- Solver action chips: GoTo / Sync / Slew to target / Nothing
- Solver mode chips: StellarSolver / Remote (INDI)
- Live telescope coordinates and solution: RA, DEC, Err, PA, Pixel scale, FOV, Focal length, F-ratio
- Solve history with color-coded dRA/dDEC (green<50", yellow<150", red beyond)
- Ekos-style target plot with concentric 50/100/150" rings
- Expandable Ekos Align log

Polar Align tab — drift-based 3-step routine:

- Capture + plate solve at 3 different RA positions
- AZ/ALT error computation from Dec drift
- Suggested screw adjustments

3.4 Capture

Multi-job sequencer in Ekos style:

- **Cooler panel** with live sensor temperature, editable target, power % bar, visual ON/OFF toggle (green when ON), and a RECONNECT DRIVER button for stuck driver recovery (e.g. ToupTek)
- **Job list** drag-and-drop reorderable, with context menu (Edit / Duplicate / Remove)
- Each job has: filter, count, exposure, gain, offset, binning, frame type (Light/Dark/Flat/Bias), transfer format (FITS/NATIVE/XISF), capture format (RAW/RGB), delay, dither flag, target name
- **+ NEW JOB**: full form with all parameters
- **Presets**: save/load JSON sequences (e.g. 'M31 LRGB night 1')
- **START SEQUENCE**: dialog with 3 options - see chapter 4

3.5 Guide

Full PHD2 control:

- Cards: RMS Total, SNR, RMS RA, RMS Dec
- Live RA/Dec tracking error chart (history of the last ~120 samples)
- Buttons: START / STOP / DITHER / FIND STAR / CALIBRATE / CLEAR CAL / PAUSE
- PHD2 equipment: version, pixel scale, calibrated, settling

4. Pre-flight pipeline

When you tap **START SEQUENCE** in Capture a dialog appears with **three execution modes**:

4.1 FULL OBSERVATION (recommended)

Runs an orchestrated 10-stage pipeline that reproduces what Ekos Scheduler does:

#	Stage	Description	Skip
1	resolve_target	Resolves the name via SIMBAD/astropy → RA/Dec	-
2	slew	Mount goto+track to the target	-
3	tracking	Waits for the mount to be Ok (max 5 min)	-
4	plate_solve	solve-field on the last frame with ±5° hint	opt
5	sync_mount	Syncs the mount on the solve, re-enables tracking	auto
6	autofocus	Iterative HFR loop with V-curve	opt
7	guide_calibrate	PHD2 clear+recalibrate, wait for Guiding (4 min)	opt
8	guide_start	PHD2 start guiding, wait for settle (3 min)	opt
9	capture_load	Loads the .esq into Ekos via DBus	-
10	capture_started	Starts the Ekos queue	-

Capture only starts after every stage has succeeded. The app shows a **live timeline** with colored stage states (grey=pending, amber=running, green=done, red=failed).

Note: Optional stages can be enabled in the Observation screen. For the first night I suggest enabling everything. On later cycles you can disable plate solve and calibrate (the slowest ones) for a faster start.

4.2 VIA EKOS (loadSequenceQueue)

Generates an `.esq` file from the Flutter jobs and loads it into the Ekos Capture queue via DBus, then starts. The sequence appears **inside** the Ekos Capture window on the desktop. Ekos handles dither, autofocus on filter change, FITS naming, meridian flip - all of its native workflow.

4.3 DIRECT (via INDI)

The app drives the INDI drivers directly without going through Ekos. Faster but Ekos won't see the sequence in its UI.

Warning: In DIRECT mode the app does NOT modify Ekos's setup. The bridge intercepts BLOBs as a parallel INDI client via enableBLOB Also, so files are not saved to disk by the bridge - everything stays in RAM.

5. Advanced screens

All available from the side drawer (tap the menu icon).

5.1 Live View

Full-screen viewer for the last captured frame, with pinch zoom and metadata (HFR, stars, exposure, filter).

5.2 Focus

Focuser control with iterative autofocus:

- Manual movement ±10/100/1000 steps in/out
- Absolute position with numeric field
- **Iterative autofocus**: set step size, n steps (odd), exposure → START. The bridge takes N shots at different positions, computes HFR for each, finds the minimum, moves to the best position.
- **Live V-curve plot** with colored points and a dashed vertical line on the best position

5.3 Align (Plate Solve)

Plate solving via solve-field astrometry.net:

- Shows the last captured frame and its metadata
- Tap **PLATE SOLVE** → the backend launches `solve-field` with hints from the current mount (5° radius), polls status every 2s
- When done, shows RA/Dec/scale extracted via `astropy.wcs.WCS` from the .wcs file
- **SYNC MOUNT** button: syncs the mount on the solve result
- Full solve-field output expandable for debug

5.4 Observatory

Dome, dust cap, flat panel, weather control:

- Weather card with all parameters (temp, humidity, wind, sky)
- Dome shutter Open/Close
- Dust cap Park/Unpark
- Flat panel toggle + 0-255 intensity slider

5.5 Scheduler

Multi-target nightly planner:

- Sky-state card: twilight phase (day/civil/nautical/astronomical/night), sun/moon altitude, lat/lon (auto-detected from the mount)
- Persistent jobs list with RA/Dec, minimum altitude, time window
- +NEW JOB: form with automatic SIMBAD target resolution
- For each job: tap ✓ to live-check conditions (twilight required, current altitude, weather safe) → dialog with the list of issues

5.6 Setup / Profiles

Active INDI driver list with **CONNECT/DISCONNECT** toggle (useful to enable drivers like XAGYL Wheel or Weather Watcher when Ekos loaded but did not connect them).

5.7 INDI Panel

Exact clone of the KStars/Ekos INDI Control Panel:

- List of every connected device
- Tap a device → all its properties grouped by Group (Main Control, Options, etc.)
- Interactive Switches (ChipToggle), editable Numbers, editable Texts, read-only Lights with colored state
- Live two-way propagation: change here → Ekos sees it; change in Ekos → it appears here
- CONNECT/DISCONNECT button at the top for each device

5.8 Files

Browser for FITS files in `~/Pictures/Ekos/`:

- Recent files list with auto-stretched thumbnails
- Tap → full-screen preview with zoom
- Light/Dark/Flat/Bias/All filters
- Multi-select with long-press and batch delete

5.9 Logs / Activity Log

Two distinct screens:

- **INDI Logs**: live INDI/Ekos message stream, filterable by module
- **Activity Log**: every HTTP request the app makes to the bridge, with millisecond timestamp, color-coded status code, duration, body. Tap for details + copy. Crucial for debugging.

5.10 Analyze

Current session timeline:

- Counters: WS events, total properties, devices
- Last frame info (object, filter, exposure, HFR, stars, size)
- Historical PHD2 RMS chart (RA in amber, Dec in cyan)
- INDI message stream, last 20 lines

6. Troubleshooting

Symptom	Likely cause	Fix
App: 'unreachable' on login	Tailscale not active on the phone or the RPi is offline	Check the Tailscale app shows Connected; ping 100.74.22.40 fro
App: 'auth_failed'	Wrong token	cat ~/.config/astroarch-bridge/token on the RPi
App sees 0 devices	Ekos profile not started	Open Ekos on the desktop and start the INDI profile
Cooler not working (POWER 0%)	toupbase driver stuck after fast on/off	Tap RECONNECT DRIVER on Capture
Sequence does not start in Ekos	UPLOAD_MODE in CLIENT (driver default)	Auto-forced to BOTH from app v0.2.2+
Plate solve fails	Astrometry index files missing or wrong scale	Check ~/.local/share/kstars/astrometry/, scale hint
WS frames does not refresh	WS state disconnected (red banner)	Manual refresh (Dashboard icon) or reconnect
FITS files not saved	UPLOAD_DIR misspelt or directory missing	App auto-creates /Pictures/Ekos/AstroarchInterface/
Multi-camera 409 error	Bridge does not know which camera to use	Use the Camera dropdown in Capture (default = primary)

6.1 Diagnostic commands on the Raspberry Pi

```
# Bridge service status
systemctl --user status astroarch-bridge

# Live log
journalctl --user -u astroarch-bridge -f

# Endpoint test
curl http://localhost:8765/healthz

# Unstuck a frozen INDI driver
indi_setprop 'CAMERA_NAME.CONNECTION.DISCONNECT=On'
sleep 2
indi_setprop 'CAMERA_NAME.CONNECTION.CONNECT=On'

# Inspect cooler state
indi_getprop 'CAMERA_NAME.CCD_TEMPERATURE.CCD_TEMPERATURE_VALUE'
indi_getprop 'CAMERA_NAME.CCD_COOLER_POWER.COOLER_POWER'
```

6.2 Diagnostics inside the app

On the Login screen there is a **TEST** button that opens the Diagnostics screen with 7 sequential steps:

- Host resolution (DNS / Tailscale)
- HTTP GET /healthz
- HTTP GET /api/system/info (auth Bearer)
- HTTP GET /api/system/snapshot (payload check)
- WebSocket /ws/state (open)
- WebSocket - first message within 5s
- WebSocket - chunked property_def reception

Each step shows the duration in milliseconds and the error details on failure. It solves 95% of issues at first try.

7. Appendix - REST API

All requests need the header `Authorization: Bearer <token>` (except /healthz). Responses are always JSON.

7.1 System

Endpoint	Method	Description
/healthz	GET	Health (no auth)
/api/system/info	GET	Bridge info (version, author)
/api/system/snapshot	GET	Global state (devices, properties, phd2, last_frame)
/api/system/connections	GET	INDI/PHD2 connection state
/api/system/devices	GET	INDI device list
/api/system/camera_roles	GET	Identifies primary vs guide camera via PHD2
/api/system/simbad	GET	?name=M31 -> RA/Dec via SIMBAD

7.2 Mount, Camera, Focuser, Filter, Guide

Endpoint	Method	Description
/api/mount/status, /goto, /park, /track, /slew, /slew_rate, /abort	GET/POST	Telescope control
/api/camera/status, /expose, /abort, /cooler, /gain, /offset, /binning, /frame_type, /transfer_format, /capture_format, /upload_setup	GET/POST	Camera control
/api/focuser/status, /abs, /rel, /abort, /autofocus, /autofocus/{id}	GET/POST	Focuser + iterative autofocus
/api/filter_wheel/status, /select	GET/POST	Filter wheel
/api/guide/status, /start, /stop, /dither, /loop, /clear_calibration, /pause, /find_star, /calibrate, /profile	GET/POST	PHD2 guide

7.3 Align, Capture/Ekos, Observation, Scheduler, Setup

Endpoint	Description
/api/align/status, /solve, /solve/{id}/sync_mount, /capture_and_solve, /ekos_capture_and_solve, /ekos_full_status	Plate solving + Ekos Align
/api/capture/ekos_alive, /ekos_run, /ekos_status, /ekos_abort, /ekos_clear	Capture via Ekos DBus
/api/observation/run, /{id}, /{id}/abort	Pre-flight pipeline 10 stages
/api/scheduler/sky_state, /jobs, /jobs/{id}/check_conditions, /weather_safe	Time scheduler
/api/setup/profiles, /active_drivers	Ekos profiles + active drivers
/api/observatory/status, /dome/shutter, /dust_cap, /flat_panel	Dome + flat panel
/api/files/recent, /preview, /download, /file (DELETE), /delete_many, /disk_usage	FITS browser

7.4 INDI panel (control panel clone)

Endpoint	Description
GET /api/indi/devices	Device list

GET /api/indi/devices/{dev}/properties	All device properties
GET /api/indi/devices/{dev}/properties/{name}	Single property
POST /api/indi/devices/{dev}/properties/{name}	Set values (Switch/Number/Text)
POST /api/indi/devices/{dev}/connect, /disconnect	Connect/disconnect driver
POST /api/indi/refresh	Force getProperties

7.5 WebSocket

URL	What it pushes
GET /ws/state?token=...	snapshot_begin -> N property_def -> snapshot_end (init), then property_def, property_set, pr
GET /ws/frames?token=...	For each frame: JSON header {type:frame_meta, size, hfr, ...} followed by JPEG bytes

8. Changelog

Version	Highlights
0.1.0	First release: 13 base screens, live WebSocket clone, Tailscale auth
0.1.4	Chunked WebSocket snapshot (fix 200KB Android payload)
0.1.5-6	HTTP/WS cleartext fix + step-by-step diagnostics
0.1.7	Camera primary/guide auto-detection via PHD2
0.1.8-10	Cooler robust toggle + capture sequence + Activity Log
0.2.0	Full Ekos clone: 8 modules + multi-job + SIMBAD + autofocus V-curve
0.2.1	Plate solving via solve-field + Time scheduler + sky_state
0.2.3	Capture via Ekos DBus (loadSequenceQueue + start)
0.2.4	FULL OBSERVATION pipeline with 10 pre-flight stages
0.2.7	Plate solve clone Ekos (full status + bin/gain configurable)
0.2.10	Plate solve UI redesign (live preview, big action button)
0.2.11	BLOB intercept zero-invasive (parallel INDI client)
0.2.12	Auto-stretch identical to KStars/Ekos (ZScale + asinh MTF)
0.2.13	Fix Ekos AlignSolverAction enum mapping (Sync/Slew/Nothing)

Future roadmap

- Mosaic planner
- Push notifications (frame done, sequence finished, weather alert)
- Possible upstream contribution to `devDucks/astroarch`

- Clear skies -
Astroarch Interface * Zarletti-Osservatorio Jupiter